

Algorithm for Planning as State Space Search

Planning Methods

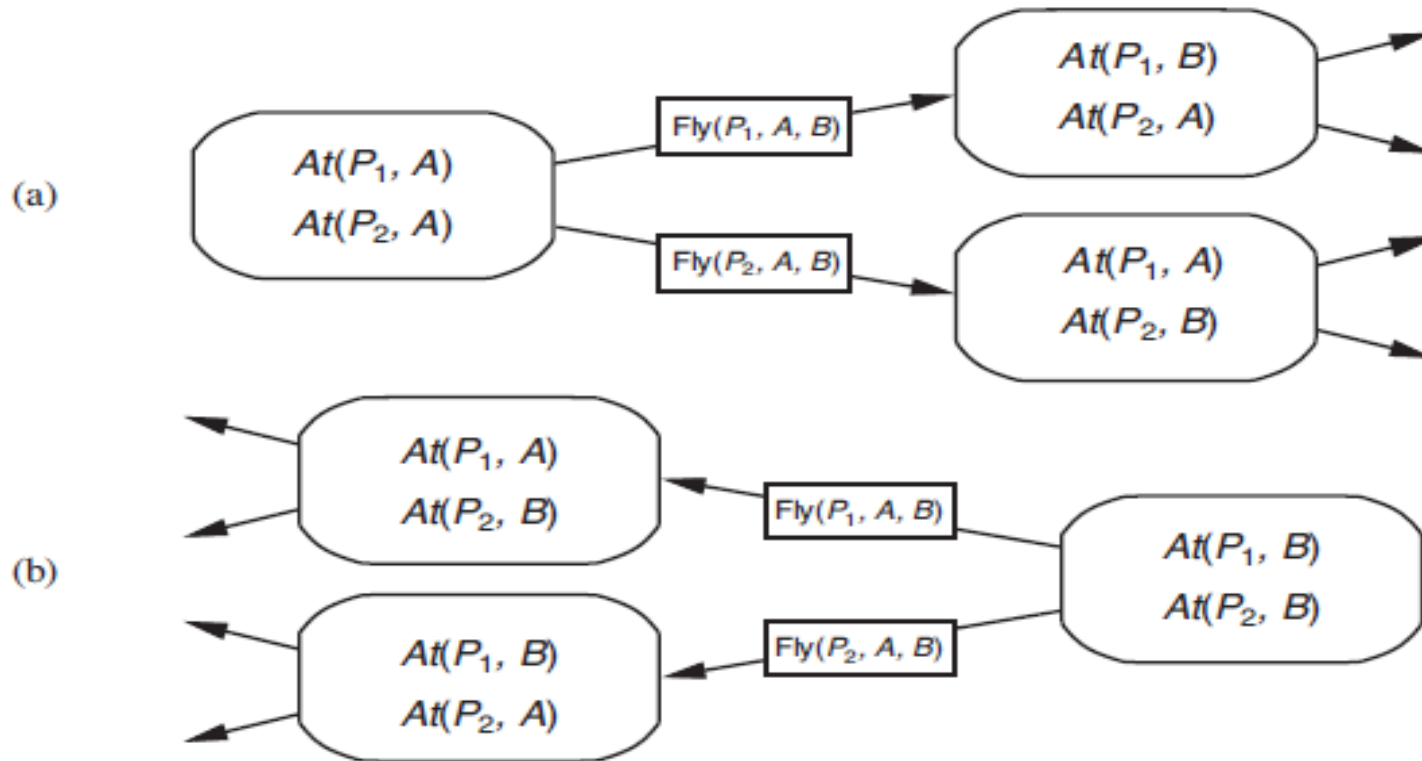
State space search

- State space consists of the initial state, set of goal states, set of actions or operations, set of states and the path cost.
- This state space needs to be searched to find a sequence of actions leading to the goal state. This can be done in the forward or backward direction.

Planning with State-space search

- The most straightforward approach of planning algorithm, is state-space search
 - Forward state-space search (Progression)
 - Backward state-space search (Regression)
- The **descriptions of actions** in a planning problem, and specify both **preconditions and effects**
- It is possible to **search in both direction**: either forward from the initial state or backward from the goal
- We can also use the explicit action and goal representations, to derive effective heuristics automatically.

Forward vs. backward search



- a. Progression Search
- b. Regression Search

Forward Search State space algorithm

Forward-search(O, s_0, g)

$s \leftarrow s_0$

$\pi \leftarrow$ the empty plan

loop

if s satisfies g then return π

$E \leftarrow \{a \mid a \text{ is a ground instance an operator in } O,$
and $\text{precond}(a)$ is true in $s\}$

if $E = \emptyset$ then return failure

nondeterministically choose an action $a \in E$

$s \leftarrow \gamma(s, a)$

$\pi \leftarrow \pi.a$

Backward Search State space

- For forward search, we started at the initial state and computed state transitions
 - new state = $\gamma(s, a)$
- For backward search, we start at the goal and compute inverse state transitions
 - new set of subgoals = $\gamma^{-1}(g, a)$
- To define $\gamma^{-1}(g, a)$, must first define *relevance*:
 - An action a is relevant for a goal g if
 - a makes at least one of g 's literals true
 - $g \cap \text{effects}(a) \neq \emptyset$
 - a does not make any of g 's literals false
 - $g^+ \cap \text{effects}^-(a) = \emptyset$ and $g^- \cap \text{effects}^+(a) = \emptyset$

Inverse State Transitions

- If a is relevant for g , then
 - $\gamma^{-1}(g, a) = (g - \text{effects}(a)) \cup \text{precond}(a)$
- Otherwise $\gamma^{-1}(g, a)$ is undefined

Backward Search State Space algorithm

Backward-search(O, s_0, g)

$\pi \leftarrow$ the empty plan

loop

if s_0 satisfies g then return π

$A \leftarrow \{a \mid a \text{ is a ground instance of an operator in } O$
and $\gamma^{-1}(g, a)$ is defined}

if $A = \emptyset$ then return failure

nondeterministically choose an action $a \in A$

$\pi \leftarrow a.\pi$

$g \leftarrow \gamma^{-1}(g, a)$

PARTIAL ORDER PLANNING (POP)

Total Order planning (TOP)

- Forward Search State space (FSSS) and Backward Search State Space (BSSS) are examples of TOP. They only explore **linear sequences of actions** from start to goal state, They **cannot take advantage** of problem **decomposition**, i.e. splitting the problem into smaller sub-problems and solving them individually.

Partial-order planning

- It works on problem decomposition. It will divide the problem into parts and achieve these sub goals independently.
- It solves the sub problems with sub plans and then combines these sub plans and reorders them based on requirements.
- In POP, ordering of the actions is partial. It does not specify which action will come first out of the two actions which are placed in the plan.

Shoe example

Init()

Goal(RightShoeOn $\wedge$ LeftShoeOn)

Action(RightSock,

 PRECOND:

 EFFECT: RightSockOn)

Action(RightShoe,

 PRECOND: RightSockOn

 EFFECT: RightShoeOn)

Action(LeftSock,

 PRECOND:

 EFFECT: LeftSockOn)

Action(LeftShoe,

 PRECOND: LeftSockOn

 EFFECT: LeftShoeOn)

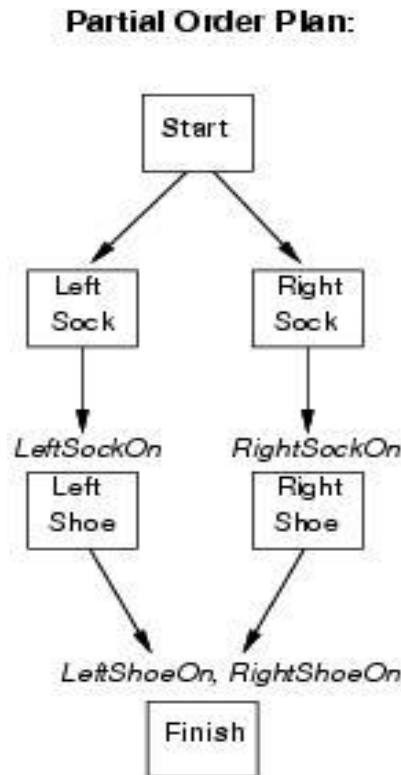
Planner: combine two action sequences

(1)leftsock, leftshoe

(2)rightsock, rightshoe

Partial-order planning(POP)

- Any planning algorithm that can place two actions into a plan without which comes first is a PO plan.



POP as a search problem

- States are (mostly unfinished) plans.
 - The empty plan contains only start and finish actions.
- Each plan has 4 components:
 - **A set of actions (steps of the plan)**
 - **A set of ordering constraints: $A < B$ (A before B)**
 - Cycles represent contradictions.
 - **A set of causal links** $A \xrightarrow{p} B$
 - The plan may not be extended by adding a new action C that conflicts with the causal link. (if the effect of C is $\neg p$ and if C could come after A and before B)
 - Also called **protection intervals** (plan P is protected between A & B)
 - **A set of open preconditions.**
 - If precondition is not achieved by action in the plan.

Example of final plan

- **Actions**= {Rightsock, Rightshoe, Leftsock, Leftshoe, Start, Finish}
- **Orderings**= {Rightsock < Rightshoe; Leftsock < Leftshoe}
- **Links** = {Rightsock->Rightsockon -> Rightshoe, Leftsock->Leftsockon-> Leftshoe, Rightshoe->Rightshoeon -> Finish, Leftshoe->Leftshoeon-> Finish}
- Open preconditions={}

POP as a search problem

- A plan is *consistent* iff there are **no cycles** in the ordering constraints and **no conflicts** with the causal links.
- A consistent plan with **no open preconditions** is a *solution*.
- A partial order plan is executed by repeatedly choosing *any* of the possible next actions.
 - This flexibility is a benefit in non-cooperative environments.

Solving POP

- Assume propositional planning problems
 - The initial plan contains *Start* and *Finish*, the ordering constraint $Start < Finish$, no causal links, all the preconditions in *Finish* are open.
 - Successor function :
 - picks one open precondition p on an action B and
 - generates a successor plan for every possible consistent way of choosing action A that achieves p .
 - Test goal

Enforcing consistency

- A **consistent plan** is a plan in which there is **no cycle** in the ordering constraints and **no conflicts** with the causal links.
- A consistent plan with **no open** preconditions is a **solution**.
- When generating successor plan:
 - The causal link $A \rightarrow p \rightarrow B$ and the ordering constraint $A < B$ is added to the plan.
 - If A is new also add $\text{start} < A$ and $A < B$ to the plan
 - Resolve conflicts between new causal link and all existing actions
 - Resolve conflicts between action A (if new) and all existing causal links.

Example: Spare tire problem

Init(*At*(Flat, Axle) $\wedge$ *At*(Spare, trunk))

Goal(*At*(Spare, Axle))

Action(*Remove*(Spare, Trunk)

PRECOND: *At*(Spare, Trunk)

EFFECT: $\neg$ *At*(Spare, Trunk) $\wedge$ *At*(Spare, Ground))

Action(*Remove*(Flat, Axle)

PRECOND: *At*(Flat, Axle)

EFFECT: $\neg$ *At*(Flat, Axle) $\wedge$ *At*(Flat, Ground))

Action(*PutOn*(Spare, Axle)

PRECOND: *At*(Spare, Ground) $\wedge$ $\neg$ *At*(Flat, Axle)

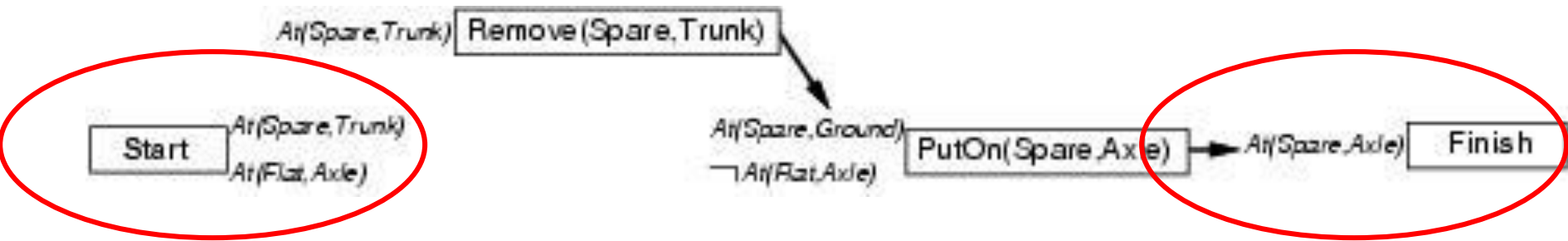
EFFECT: *At*(Spare, Axle) $\wedge$ $\neg$ *Ar*(Spare, Ground))

Action(*LeaveOvernight*

PRECOND:

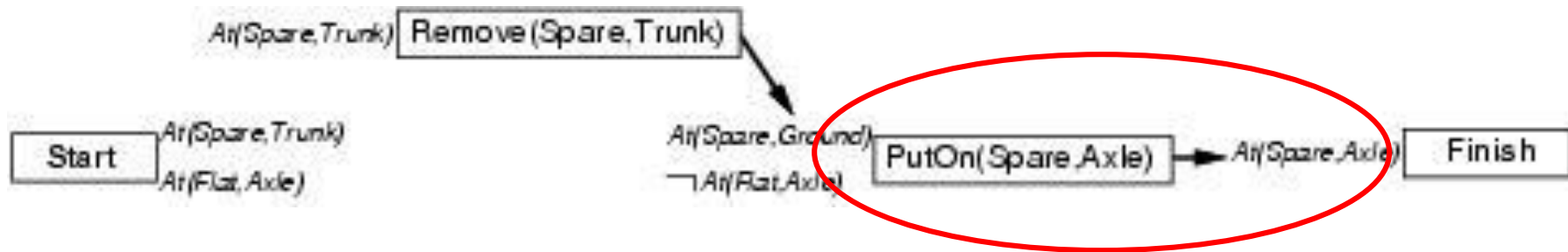
EFFECT: $\neg$ *At*(Spare, Ground) $\wedge$ $\neg$ *At*(Spare, Axle) $\wedge$ $\neg$ *At*(Spare, trunk) $\wedge$ $\neg$ *At*(Flat, Ground) $\wedge$ $\neg$ *At*(Flat, Axle))

Solving the problem



- Initial plan: Start with EFFECTS and Finish with PRECOND.

Solving the problem



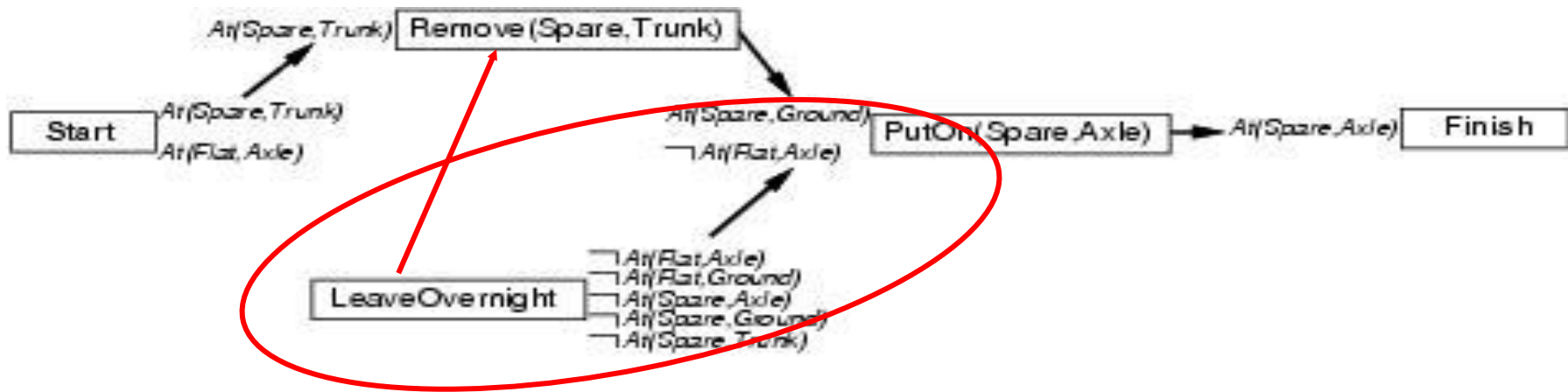
- Initial plan: Start with EFFECTS and Finish with PRECOND.
- Pick an open precondition: $At(Spare, Axle)$
- Only $PutOn(Spare, Axle)$ is applicable
- Add causal link: $PutOn(Spare, Axle) \xrightarrow{At(Spare, Axle)} Finish$
- Add constraint : $PutOn(Spare, Axle) < Finish$

Solving the problem



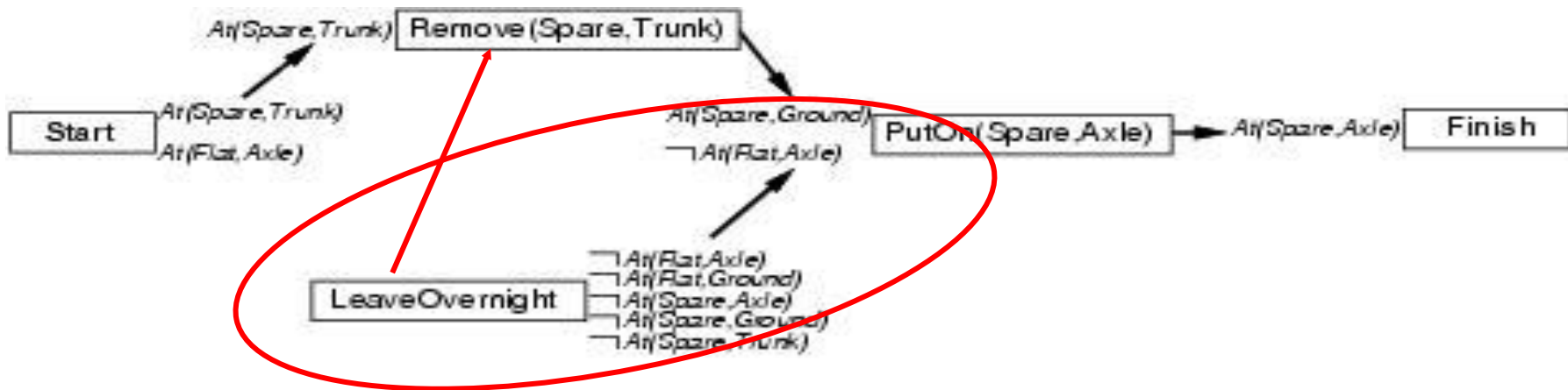
- Pick an open precondition: $At(Spare, Ground)$
- Only $Remove(Spare, Trunk)$ is applicable
- Add causal link: $Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle)$
- Add constraint : $Remove(Spare, Trunk) < PutOn(Spare, Axle)$

Solving the problem



- Pick an open precondition: $\neg At(Flat, Axle)$
- *LeaveOverNight* is applicable
- conflict: *LeaveOverNight* also has the effect $\neg At(Spare, Ground)$
- $Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle)$
- To resolve, add constraint : $LeaveOverNight < Remove(Spare, Trunk)$

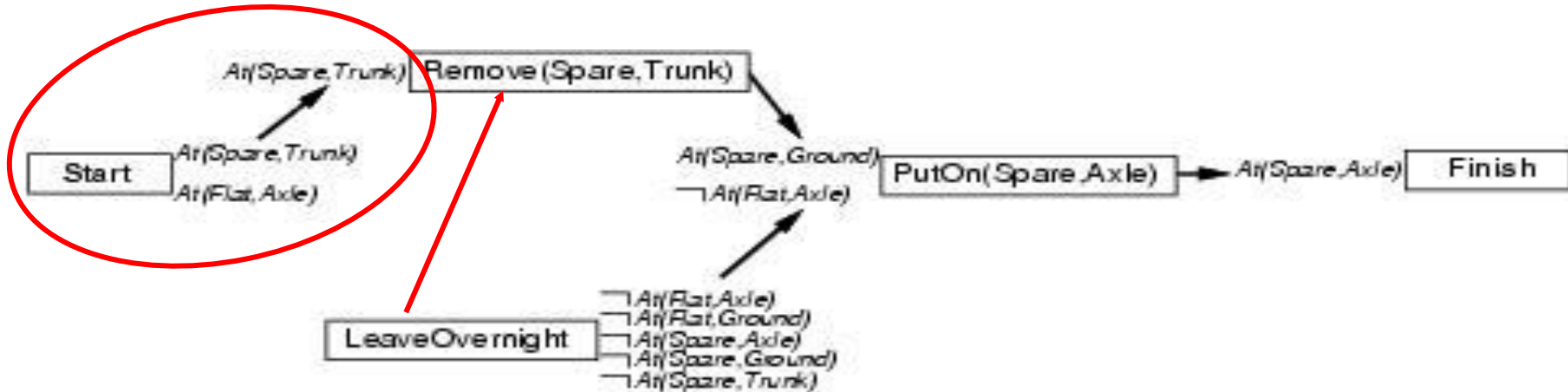
Solving the problem



- Pick an open precondition: $At(Spare, Ground)$
- *LeaveOverNight* is applicable
- conflict: $Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle)$
- To resolve, add constraint : $LeaveOverNight < Remove(Spare, Trunk)$
- Add causal link:

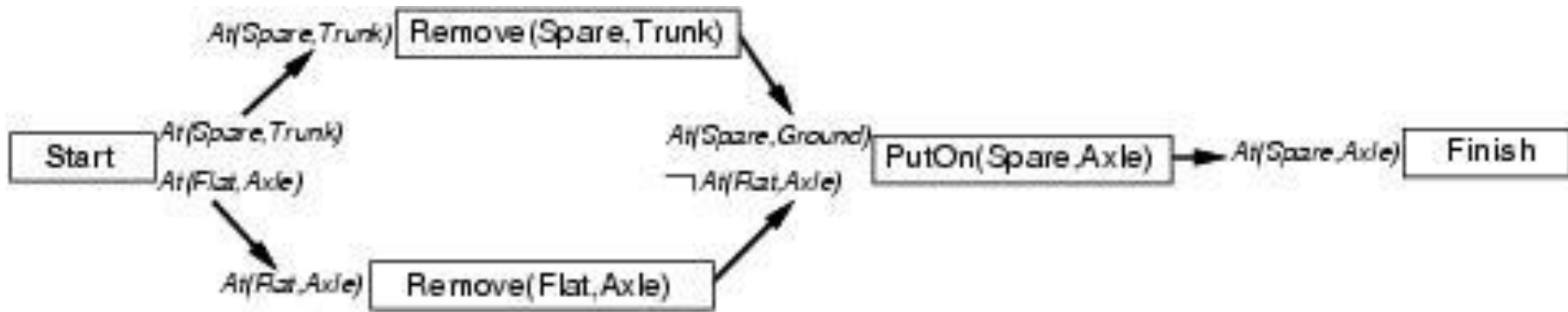
$$LeaveOverNight \xrightarrow{\neg At(Flat, Axle)} PutOn(Spare, Axle)$$

Solving the problem



- Pick an open precondition: $At(Spare, Trunk)$
- Only *Start* is applicable
- Add causal link: $Start \xrightarrow{At(Spare, Trunk)} Remove(Spare, Trunk)$
- Conflict: of causal link with effect $At(Spare, Trunk)$ in *LeaveOverNight*
 - No re-ordering solution possible.
- backtrack

Solving the problem



- Remove *LeaveOverNight*, *Remove(Spare, Trunk)* and causal links
- Repeat step with *Remove(Spare, Trunk)*
- Add also *Remove(Flat, Axle)* and finish

PLANNING & ACTING IN NONDETERMINISTIC DOMAINS

INTRODUCTION

- Classical planning is **the problem of finding a sequence of actions for achieving a goal from an initial state assuming that actions have deterministic effects.**
- In case of Classical Planning, the environment is **fully observable, deterministic, static and discrete.**
- The action descriptions are **correct & complete.** In this circumstance, an agent can plan & then execute the plan “**with its eyes close**”.
- In an **uncertain environment**, an agent must use its precepts to discover what is happening while the plan is being executed and possibly modify or replace the plan if something goes wrong (unexpected things happened)
- So the agents have to deal with both ***incomplete & incorrect*** information
- Both incomplete & incorrectness arise because the world is **partially observable, non-deterministic, or both.**

- The possibility of having complete or correct knowledge depends on how much indeterminacy there is in the world.
- Two types of indeterminacy
 - Bounded indeterminacy
 - Unbounded indeterminacy
- **Bounded indeterminacy:** actions can have unpredictable effects, but the possible effects can be listed in the action description axioms (Eg: flipping a coin)
- **Unbounded indeterminacy:** the set of possible preconditions or effects is either unknown or too large to be enumerated completely. (It is closely related to the qualification problem)
- In this case, the domain will be complex and dynamic (automatic taxi driving, economic planning & military strategy)

- There are four planning methods for handling indeterminacy
 - » (1). **Sensorless planning**
 - » (2). **Conditional planning**
 - » (3). **Execution monitoring & replanning**
 - » (4). **Continuous planning**
- The (1) and (2) are suitable for bounded indeterminacy and (3) and (4) for unbounded indeterminacy.
- **Sensorless planning**: Handles domains where the state of the world is **not fully known**. It is also called **conformant planning**. This method constructs standard, **sequential plans that are to be executed without perception**.
- This approach must ensure that the plan achieves the goal in all possible circumstances, regardless of the true initial state & actual outcomes.
- It relies on **coercion**(force/demand): The idea that the world can be forced into a given state even when the agent has only partial information about the current state.
- Coercion is not always possible, hence this approach is not applicable all the time.

- **Conditional planning:** Also called **contingency planning**. It deals with bounded indeterminacy by constructing a conditional plan with different branches for the different contingencies that could arise.
- The agent finds out **which part of the plan to execute by including sensing action in the plan to test for the appropriate conditions.**

Conditional plans require conditional steps:

□ **If <test> then *plan_A* else *plan_B***

if AtL ∧ CleanL then Right else Suck

□ **Plans become trees**

- **Execution monitoring & replanning:** The agent can **use any of the preceding planning techniques** (classical, sensorless, or conditional) to construct a plan, but it also uses execution monitoring to judge whether the plan has a provision for the **actual current situation** or needs to be revised.
- Replanning occurs when something goes wrong.

- **Continuous planning:** This planner is designed to persist over a lifetime.
- It can handle unexpected circumstances in the environment, even if **these occur while the agent is in the middle of constructing a plan.**
- It can handle the abandonment of goals and the creation of additional goals by goal formulation.

CONDITIONAL PLANNING

Conditional Planning

- The agent finds out which part of the plan to execute by including sensing actions in the plan to test for the appropriate conditions.
- **Contingency planning:** incomplete planning by constructing a conditional plan that accounts for each possible situation.
- **Sensing action:** The agent includes sensing actions to find which part of the plan to be executed.
- **For example:** The **shopping agent** might want to include a sensing action in its shopping plan to check the price of some object in case it is too expensive.

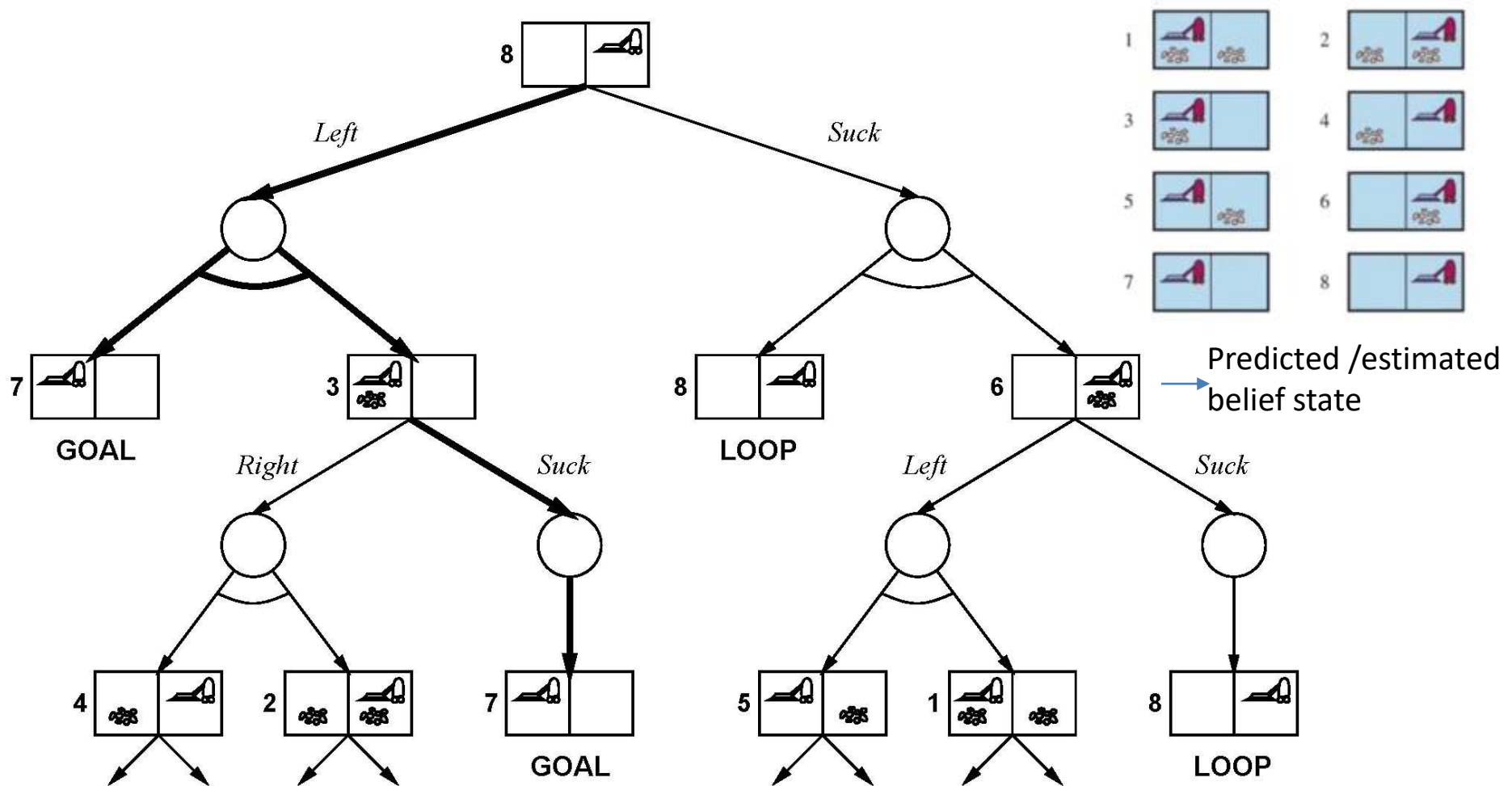
Three kind of Environments

- Fully Observable
 - The agent always knows the current state.
- Partially Observable
 - The agent knows only a certain amount about the actual state. (much more common in the real world)
- Unknown
 - The agent knows nothing about the current state.

CP in Fully Observable Environments (FOE)

- The agent used conditional steps to check the state of the environment to decide what to do next.
- Plan information stored in a library
 - Ex: Action(Left) -> Clean v Right (disjunction)
- Syntax:
 - If <test> then plan_A else plan_B

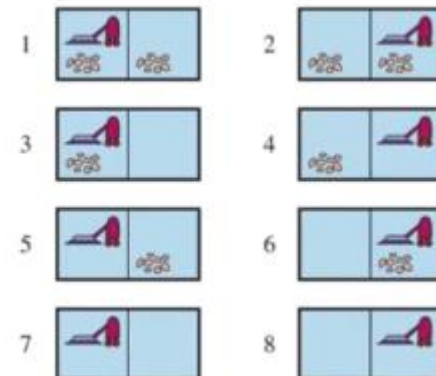
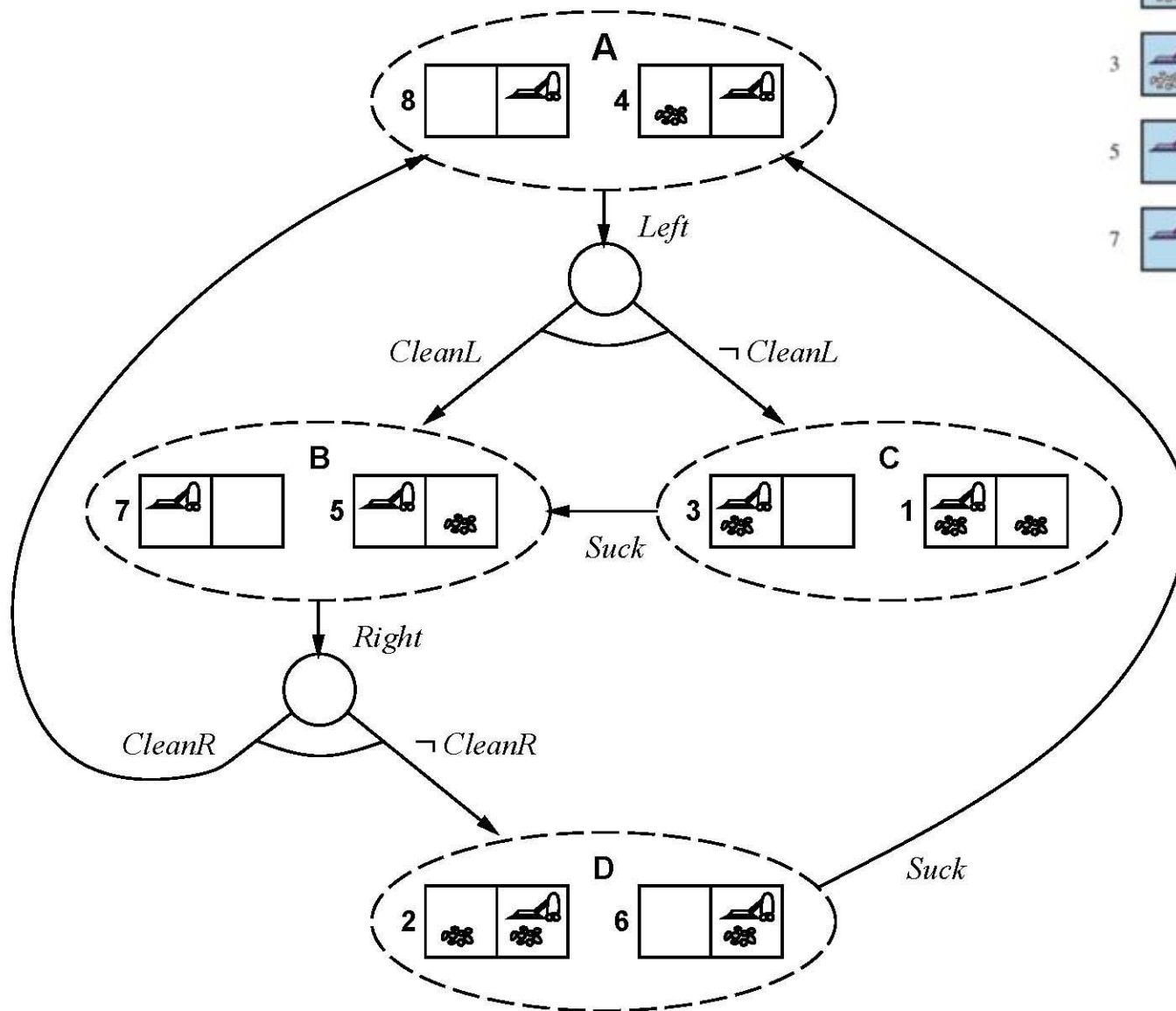
The search tree for the vacuum world



State nodes (squares) and chance nodes (circles)

CP in Partially Observable Environments (POE)

- The agent has limited information about the environment.
- Uncertain actions and imperfect sensors
- Initial state is a state set - a **belief state**
- Sensing is important!
 - **Automatic sensing**
 - At every step, the agent gets all the available percepts
 - **Active sensing**
 - Percepts are obtained through the execution of specific sensory actions.
 - checkDirt and checkLocation
- Determine “both squares are clean” with local dirt sensing
 - The vacuum agent is AtR and knows about R, how about L?
 - Dirt can sometimes be left behind when the agent leaves a clean square



Hierarchical Planning

- Hierarchical planning in artificial intelligence (AI) is a planning approach that involves organizing tasks and actions into multiple levels of abstraction or hierarchy, where higher-level tasks are decomposed into a sequence of lower-level tasks.
- It provides a way to efficiently reason and plan in complex domains by utilizing a hierarchy of goals and subgoals. Hierarchical planning allows AI systems to make decisions at different levels of abstraction, enabling them to tackle complex tasks and environments effectively.

Components of Hierarchical Planning

Hierarchical planning in artificial intelligence (AI) typically involves several key components, including:

High-level goals: The overall objectives or tasks that the AI system aims to achieve.

Task decomposition: Breaking down high-level goals into lower-level tasks or subgoals.

Planning hierarchy: The organization of tasks or subgoals into a hierarchical structure, such as a tree or a directed acyclic graph (DAG).

Plan generation at different levels: Reasoning and planning at different levels of the hierarchy, with plans generated for achieving subgoals or actions.

Plan synthesis: Combining the plans for achieving subgoals or actions into a cohesive plan for execution.

Plan execution: Carrying out the actions or subgoals in the plan in the correct order.

Plan adaptation: Revising plans at different levels of abstraction to accommodate changes in the environment or goals

Hierarchical Decomposition

- High-Level Abstraction
 - reducing a high-level operator to a set of lower-level ones
 - not able to directly feed into agents effectors.
 - Also called operator reduction, operator expansion
- Low-Level Abstraction
 - too much time and steps.
- Abstract Operator - incrementally decomposes a planning problem.
- Primitive Operator - represents executable actions.
- This kind of planning is also called *hierarchical task network planning*

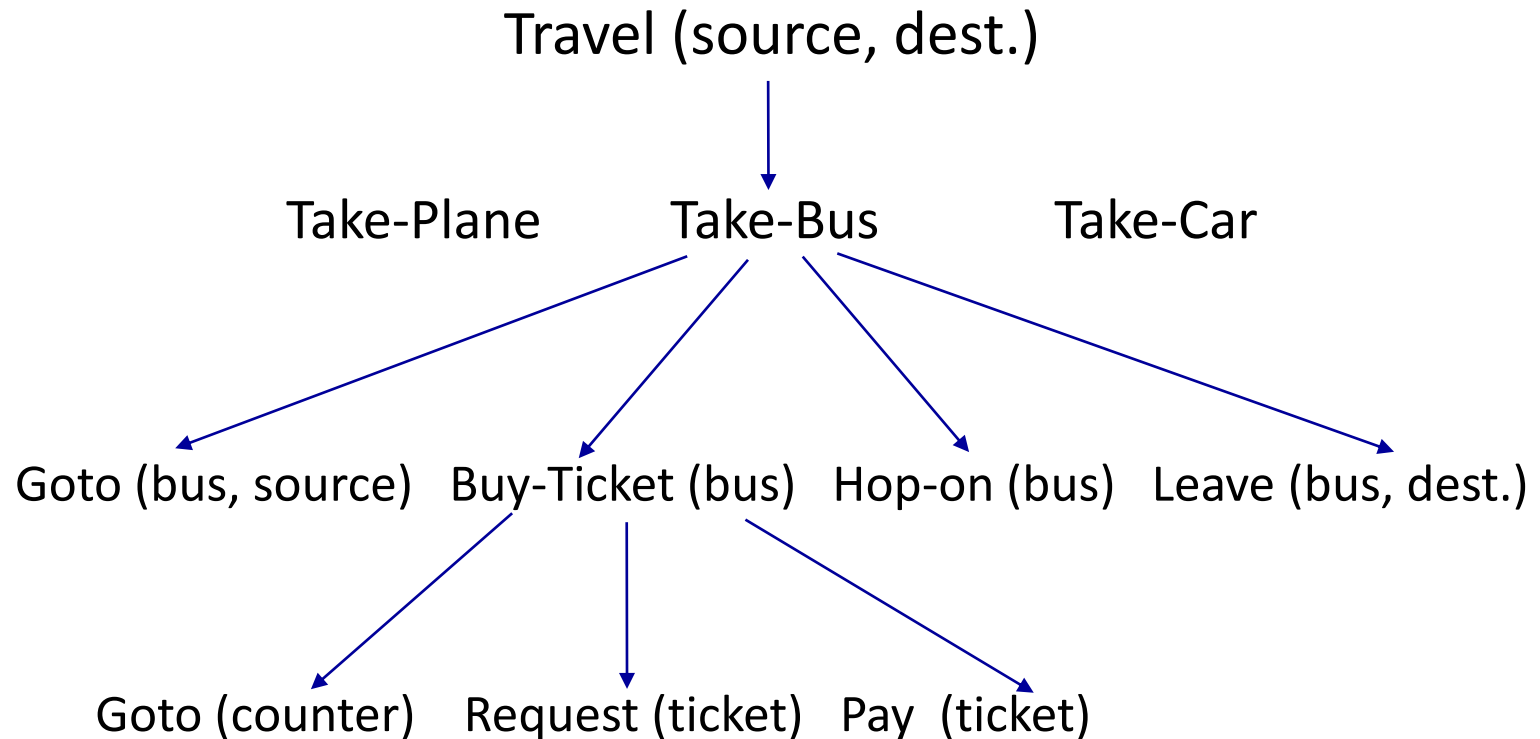
Hierarchical Planning

Hierarchical Planning / Plan Decomposition

Plans are organized in a hierarchy. Links between nodes at different levels in the hierarchy denote a **decomposition** of a “**complex action**” into more **primitive actions** (**operator expansion**).

The lowest level corresponds to executable actions of the agent.

Hierarchical Plan - Example



Example

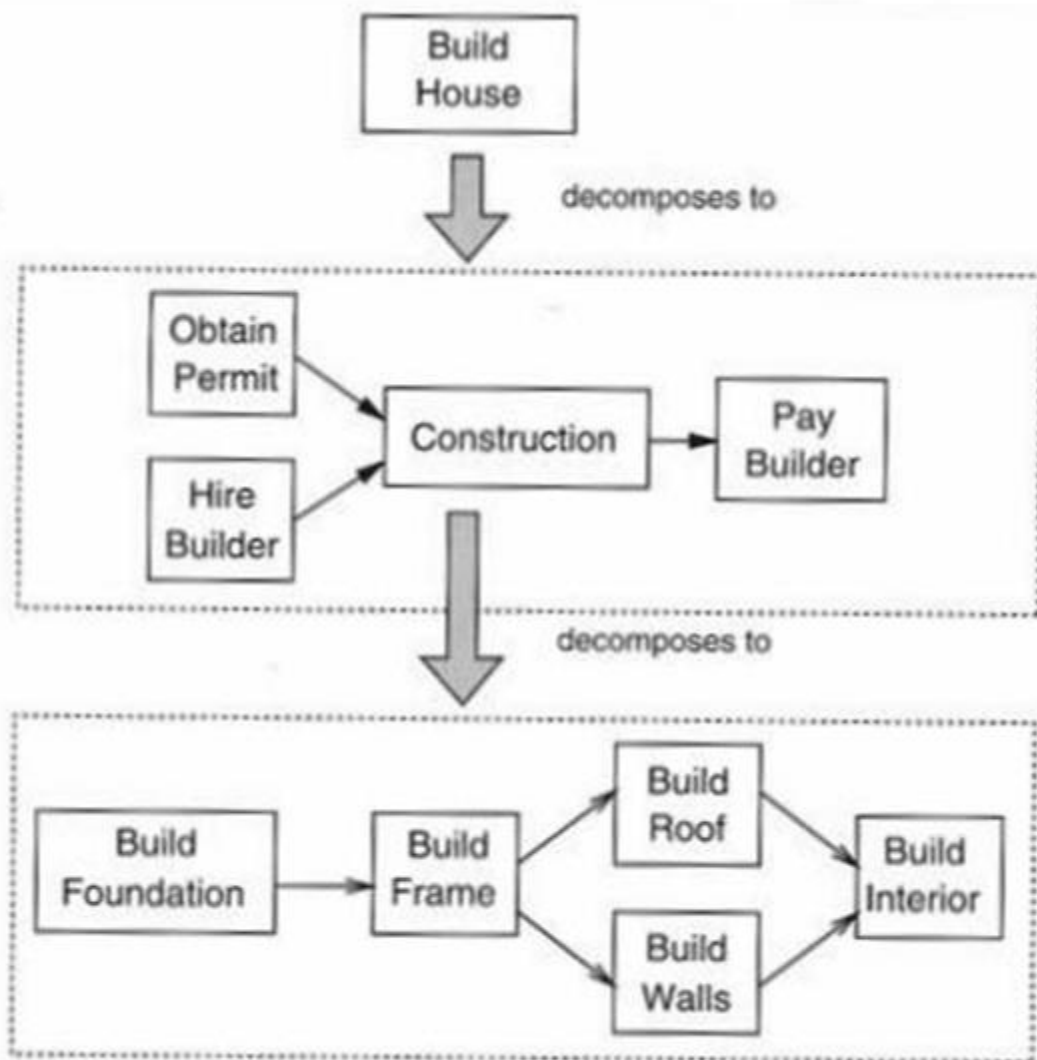
- Planning for "Going to Goa this Christmas"
 - Major Steps :
 - Hotel Booking
 - Ticket Booking
 - Reaching Goa
 - Staying and enjoying there
 - Coming Back
 - Minor Steps :
 - Take a taxi to reach station / airport
 - Have candle light dinner on beach
 - Take photos

Example

- 1st level Plan :
 - Buy Ticket (3), Reach airport(3), Reach Goa(3)
- 2nd level Plan :
 - Finding flight (2), Buy Ticket (3), Get taxi(2), Reach airport(3), Boarding plane(2), Reach Goa(3)
- 3rd level Plan (final) :
 - Opening makemytrip.com (1), Finding flight (2), Buy Ticket (3), Get taxi(2), Reach airport(3), Pay-driver(1), Check in(1), Boarding plane(2), Reach Goa(3)

Example

- Actions required for “Travelling to Goa”:
 - Opening makemytrip.com (1)
 - Finding flight (2)
 - Buy Ticket (3)
 - Get taxi(2)
 - Reach airport(3)
 - Pay-driver(1)
 - Check in(1)
 - Boarding plane(2)
 - Reach Goa(3)



Hierarchical decomposition of the operator *Build House* into a partial-order plan.

Analysis of Hierarchical Decomposition

- Subroutines are good
 - can be combined to create large plans.
- Abstract solution
- Two solutions
 - downward solution
 - upward solution
- A simplified model
- Assume that there is one solution with n primitive steps.
- Time to choose right steps.